

host_agent

Initialised

planning_agent

Initialised

coding_agent

Initialised

plotting_agent

Initialised

address_agent

Initialised

named_area_agent

Initialised

buildings_agent

Initialised

water_features_agent

Initialised

water_network_agent

Initialised

land_features_agent

Initialised

land_use_features_agent

Initialised

structures_agent

Initialised

human

Initialised

Conversation Log

human

Show me all areas of water in Exeter?

host_agent

Tool call: send_message - target: planning_agent - task_description: User wants to see all areas of water in Exeter. Provide the general steps to solve this query.

planning_agent

The area specified is Exeter, so there will be 1 entry for Exeter. The entity to search for is areas of water, which could include rivers, lakes, and other bodies of water. Since the user wants to see all areas of water, we will look for as many entries as possible within Exeter.

output steps: ["Find Exeter 1 area", "Search for areas of water in Exeter as many search results"]

host_agent

Tool call: send_message - target: named_area - task_description: Find the area polygon for Exeter.

named_area_agent

Tool call: call_os_ngd - filters: None - bbox: None - polygon_or_point: True - street_address: None - filename: exeter_area_polygon

Tool call result: Search results have been found. Artifact generated is : exeter_area_polygon, Description is : A geopandas dataframe containing named area data with no filters applied for the query (so remember no search is performed here so spurious entities present) Find the area polygon for Exeter..., count of records fetched : 98.

Tool call: send_message - target: data_analysis_agent - task_description: Filter the artifact 'exeter_area_polygon' to find the specific polygon for Exeter. Save the filtered results as 'filtered_exeter_area_polygon'.

data_analysis_agent

Tool Code Executor

```
def filter_exeter_area_polygon(data: list):
    gdf = data[0]
    # Filter for Exeter using multiple relevant columns
    filtered = gdf[((gdf['name1_text'].str.lower() == 'exeter') |
                    (gdf['name1_text'].str.lower().str.contains('exeter')) |
                    (gdf['description'].str.lower() == 'city')) &
                  (gdf['descriptiongroup'].str.lower().isin(['settlement', 'part of settlement']))]
    summary = f"Filtered to {len(filtered)} polygon(s) for Exeter using name and description columns."
    artifact_name = 'filtered_exeter_area_polygon'
    artifact_description = 'GeoDataFrame containing only the polygon(s) representing Exeter, filtered by name and settlement/city description.'
    return [summary, artifact_name, artifact_description, filtered]
```

Filtered to 1 polygon for Exeter using the name and description columns. The result is saved as 'filtered_exeter_area_polygon', which contains only the polygon(s) representing Exeter, filtered by name and settlement/city description. If you need further details or analysis on this filtered polygon, let me know!

named_area_agent

The area polygon for Exeter has been successfully filtered and saved as `filtered_exeter_area_polygon`. This GeoDataFrame contains only the polygon(s) representing Exeter, filtered by name and settlement/city description.

host_agent

Tool call: send_message - target: water_features - task_description: Search for all areas of water within the artifact 'filtered_exeter_area_polygon'.

water_features_agent

Tool call: call_os_ngd - filters: None - bbox: filtered_exeter_area_polygon - polygon_or_point: True - street_address: False - filename: exeter_water_features

Tool call result: Multiple search results have been found in multiple datasets. A summary of each is provided. Artifacts generated are : ['waterpoint_exeter_water_features', 'water_exeter_water_features'], Descriptions are : ['A geopandas dataframe containing water point data with bbox applied for the query {query} with filters applied {filters}', 'A geopandas dataframe containing water data with bbox applied for the query {query} with filters applied {filters}'], counts of records fetched are : [265, 131].

I found two datasets related to water features within the 'filtered_exeter_area_polygon':

- 1. **waterpoint_exeter_water_features**: Contains 265 records of water point data.
- 2. **water_exeter_water_features**: Contains 131 records of water data.

If you need further analysis or filtering, please let me know!

host_agent

Tool call: send_message - target: water_network - task_description: Search for all water network features (rivers, streams, lakes, etc.) within the artifact 'filtered_exeter_area_polygon'.

water_network_agent

Tool call: call_os_ngd - filters: None - bbox: filtered_exeter_area_polygon - polygon_or_point: True - street_address: False - filename: exeter_water_network_features

Tool call result: Multiple search results have been found in multiple datasets. A summary of each is provided. Artifacts generated are : ['waterlinkset_exeter_water_network_features', 'waterlink_exeter_water_network_features'], Descriptions are : ["A geopandas dataframe containing water link set data with bbox applied for the query Search for all water network features (rivers, streams, lakes, etc.) within the artifact 'filtered_exeter_area_polygon'. with no filters applied for the query (so remember no search is performed here so spurious entities present).", "A geopandas dataframe containing water link data with bbox applied for the query Search for all water network features (rivers, streams, lakes, etc.) within the artifact 'filtered_exeter_area_polygon'. with no filters applied for the query (so remember no search is performed here so spurious entities present)."], counts of records fetched are : [11, 430].

Tool call: send_message - target: data_analysis_agent - task_description: Analyze the artifact 'waterlinkset_exeter_water_network_features' to filter out spurious entities and provide a summary of valid water network features. Save the results as 'filtered_waterlinkset_exeter'.

data_analysis_agent

Tool Code Executor

```
def filter_valid_water_network_features(data: list):  
    import geopandas as gpd  
    import pandas as pd
```

```

gdf = data[0]
# Criteria for valid water network features:
# - Non-null and non-empty 'name1_text'
# - 'description' should be one of the valid types
# - 'geometry' should not be null
# - 'theme' should be 'Water'
valid_descriptions = [
    'Named Canal',
    'Named Stretch Of Inland Water',
    'Named Watercourse'
]
filtered = gdf[
    (gdf['name1_text'].notnull()) &
    (gdf['name1_text'].str.strip() != '') &
    (gdf['description'].isin(valid_descriptions)) &
    (gdf['geometry'].notnull()) &
    (gdf['theme'] == 'Water')
].copy()
summary = f"Filtered valid water network features: {len(filtered)} out of {len(gdf)} total entries."
artifact_name = 'filtered_waterlinkset_exeter'
artifact_description = (
    "A GeoDataFrame containing only valid water network features from the Exeter dataset. "
    "Entries have non-null names, valid descriptions (canal, stretch of inland water, watercourse), "
    "non-null geometry, and are themed as 'Water'."
)
return [summary, artifact_name, artifact_description, filtered]

```

Filtered valid water network features: 11 out of 11 total entries. An artifact has been generated with the name: filtered_waterlinkset_exeter and description: A GeoDataFrame containing only valid water network features from the Exeter dataset. Entries have non-null names, valid descriptions (canal, stretch of inland water, watercourse), non-null geometry, and are themed as 'Water'.

host_agent

Tool call: send_message - target: data_analysis_agent - task_description: Analyze the artifact 'waterlink_exeter_water_network_features' to filter out spurious entities and provide a summary of valid water network features. Save the results as 'filtered_waterlink_exeter'.

data_analysis_agent

Tool Code Executor


```
def filter_valid_water_network_features(data: list):
    import geopandas as gpd
    import pandas as pd

    gdf = data[0]
    # Valid water network features: theme is 'Water', description is a valid type, geometry is not null
    valid_descriptions = [
        'Watercourse', 'Canal', 'Leat', 'Still Water', 'Drain', 'Exeter Canal', 'Brook', 'Stream', 'River'
    ]
    # Filter for valid theme, valid description, and non-null geometry
    filtered = gdf[
        (gdf['theme'] == 'Water') &
        (gdf['description'].isin(valid_descriptions)) &
        (gdf['geometry'].notnull())
    ].copy()

    summary = f"Filtered valid water network features: {len(filtered)} out of {len(gdf)} total entries."
    artifact_name = 'filtered_waterlink_exeter'
    artifact_description = (
        "A GeoDataFrame containing only valid water network features from the Exeter dataset. "
        "Entries have valid descriptions (Watercourse, Canal, Leat, Still Water, Drain, Exeter Canal, Brook, Stream, River), "
        "non-null geometry, and are themed as 'Water'."
    )
    return [summary, artifact_name, artifact_description, filtered]
```

Filtered valid water network features: 430 out of 430 total entries. An artifact has been generated with the name: filtered_waterlink_exeter and description: A GeoDataFrame containing only valid water network features from the Exeter dataset. Entries have valid descriptions (Watercourse, Canal, Leat, Still Water, Drain, Exeter Canal, Brook, Stream, River), non-null geometry, and are themed as 'Water'.

water_network_agent

The search and filtering process for water network features within the 'filtered_exeter_area_polygon' has been completed. Here are the results:

- Filtered Artifacts:**
- filtered_waterlinkset_exeter:** Contains 11 valid water network features. These entries have non-null names, valid descriptions (canal, stretch of inland water, watercourse), non-null geometry, and are themed as 'Water'.
- filtered_waterlink_exeter:** Contains 430 valid water network features. These entries have valid descriptions (Watercourse, Canal, Leat, Still Water, Drain, Exeter Canal, Brook, Stream, River), non-null geometry, and are themed as 'Water'.

These artifacts provide a comprehensive view of the valid water network features in the specified area.

host_agent

Tool call: send_message - target: plotting_agent - task_description: Plot all areas of water in Exeter using the following artifacts: 'filtered_exeter_area_polygon', 'water_exeter_water_features', and 'filtered_waterlink_exeter'. Show all water bodies and water network features within the Exeter area.

plotting_agent

Tool Code Executor

```
def plot_exeter_water_areas(data: list):
    import geopandas as gpd
    import folium
    import pandas as pd
    from shapely.geometry import mapping

    # Unpack data
    exeter_area = data[0]
    water_features = data[1]
    water_network = data[2]

    # Ensure all CRS are EPSG:4326 for folium
    exeter_area = exeter_area.to_crs(epsg=4326)
    water_features = water_features.to_crs(epsg=4326)
    water_network = water_network.to_crs(epsg=4326)

    # Clip water features and water network to Exeter area
    exeter_polygon = exeter_area.iloc[0].geometry
    water_features_clipped = water_features[water_features.geometry.within(exeter_polygon)]
    water_network_clipped = water_network[water_network.geometry.within(exeter_polygon)]

    # Remove non-serializable columns (like Timestamps) for folium GeoJson
    def remove_nonserializable_columns(gdf):
        cols_to_keep = []
        for col in gdf.columns:
            if col == 'geometry':
                cols_to_keep.append(col)
            elif pd.api.types.is_string_dtype(gdf[col]) or pd.api.types.is_numeric_dtype(gdf[col]) or pd.api.types.is_bool_dtype(gdf[col]):
                cols_to_keep.append(col)
        return gdf[cols_to_keep]

    water_network_clipped = remove_nonserializable_columns(water_network_clipped)
    exeter_area_simple = remove_nonserializable_columns(exeter_area)
```



```

# Create folium map centered on Exeter
centroid = exeter_area.geometry.centroid.iloc[0]
m = folium.Map(location=[centroid.y, centroid.x], zoom_start=13, tiles='cartodbpositron')

# Plot Exeter area boundary
folium.GeoJson(
    exeter_area_simple,
    name='Exeter Area',
    style_function=lambda x: {'fillColor': '#00000000', 'color': 'black', 'weight': 2}
).add_to(m)

# Plot water features (points)
for idx, row in water_features_clipped.iterrows():
    folium.CircleMarker(
        location=[row.geometry.y, row.geometry.x],
        radius=5,
        color='blue',
        fill=True,
        fill_color='blue',
        fill_opacity=0.7,
        popup=row['description'] if 'description' in row else None
    ).add_to(m)

# Plot water network (lines/polygons)
folium.GeoJson(
    water_network_clipped,
    name='Water Network',
    style_function=lambda x: {'color': 'deepskyblue', 'weight': 3}
).add_to(m)

# Add layer control
folium.LayerControl().add_to(m)

# Save map
map_filename = 'exeter_water_areas_map.html'
m.save(map_filename)

# Prepare summary
summary = (
    f"Plotted all areas of water in Exeter. "
    f"Water features (points): {len(water_features_clipped)}. "
    f"Water network features (lines/polygons): {len(water_network_clipped)}. "
    f"First 5 water features: {water_features_clipped[['description']].head(5).to_dict('records')} "
    f"First 5 water network features: {water_network_clipped[['description']].head(5).to_dict('records')}"
)

return [

```

```
summary,
'exeter_water_areas_map',
'A folium map showing all water bodies and water network features within the Exeter area. Water features are shown as blue points, water network as blue lines, and the Exeter boundary is outlined in black.
map_filename
]
```

Plotted all areas of water in Exeter. Water features (points): 128. Water network features (lines/polygons): 400. First 5 water features: [{ 'description': 'Culvert Entrance'}, { 'description': 'Culvert Entrance'}, { 'description': 'Culvert Entrance'}, { 'description': 'Culvert Entrance'}, { 'description': 'Culvert Entrance'}] First 5 water network features: [{ 'description': 'Watercourse'}, { 'description': 'Drain'}, { 'description': 'Drain'}, { 'description': 'Watercourse'}, { 'description': 'Watercourse'}]

plotting_agent

Plotted all areas of water in Exeter. Water features (points): 128. Water network features (lines/polygons): 400.

First 5 water features: - { 'description': 'Culvert Entrance'} - { 'description': 'Culvert Entrance'} - { 'description': 'Culvert Entrance'} - { 'description': 'Culvert Entrance'} - { 'description': 'Culvert Entrance'}

First 5 water network features: - { 'description': 'Watercourse'} - { 'description': 'Drain'} - { 'description': 'Drain'} - { 'description': 'Watercourse'} - { 'description': 'Watercourse'}

Artifact generated:

- Name: exeter_water_areas_map
- Description: A folium map showing all water bodies and water network features within the Exeter area. Water features are shown as blue points, water network as blue lines, and the Exeter boundary is outlined in black.
- Data: exeter_water_areas_map.html

host_agent

All areas of water in Exeter have been mapped for you. This includes:

- Water features (such as lakes, ponds, and culvert entrances) shown as blue points.
- Water network features (such as rivers, streams, canals, and drains) shown as blue lines.
- The boundary of Exeter is outlined in black.

A total of 128 water features and 400 water network features have been identified and plotted within the Exeter area.

If you would like to view or download the map, let me know! If you need more details about specific water bodies or want to focus on a particular type (e.g., only rivers or lakes), please specify.